```
bc4278_nyu_edu@nyu-dataproc-m:~/6_HW$ spark-shell --master yarn --deploy-mode client --conf
spark.dataset.path=6_HW/cleaned < CountRecs.scala
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/05 06:48:43 INFO SparkEnv: Registering MapOutputTracker
26/04/05 06:48:43 INFO SparkEnv: Registering BlockManagerMaster
26/04/05 06:48:43 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/04/05 06:48:43 INFO SparkEnv: Registering OutputCommitCoordinator
Spark context Web UI available at http://nyu-dataproc-m.c.hpc-dataproc-19b8.internal:36033
Spark context available as 'sc' (master = yarn, app id = application_1768446951072_8977).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.3
      /_/

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 11.0.20.1)
Type in expressions to have them evaluated.
Type :help for more information.

scala> import org.apache.spark.sql.functions._
import org.apache.spark.sql.functions._

scala> val hdfsBase = spark.conf.getOption("spark.dataset.path").getOrElse("6_HW/ingested")
hdfsBase: String = 6_HW/cleaned

scala> println(s">> Profiling data from: ${hdfsBase}")
>> Profiling data from: 6_HW/cleaned
```

```
scala> // 1. CORRIDORS

scala> println("== CORRIDORS ==")
== CORRIDORS ==

scala> val corridors = spark.read.option("header","true").option("inferSchema","true").csv(s"$
{hdfsBase}/corridors")
corridors: org.apache.spark.sql.DataFrame = [year: int, origin_region: string ... 3 more field
s]

scala> println(s"Total records: ${corridors.count()}")
Total records: 4863

scala> corridors.printSchema()
root
 |-- year: integer (nullable = true)
 |-- origin_region: string (nullable = true)
 |-- destination_region: string (nullable = true)
 |-- metric: string (nullable = true)
 |-- value: double (nullable = true)


scala> corridors.rdd.map(r => (r.getAs[String]("metric"), 1)).reduceByKey(_ + _).collect().sor
tBy(-_._2).foreach { case (k,v) => println(s"  $k -> $v") }
  transport_operations_total -> 1257
  tonne_km_millions -> 1255
  tonnes_thousands -> 1255
  transport_operations_empty -> 1096

scala> corridors.select("year").distinct().orderBy("year").show(50,false)
+----+
|year|
+----+
|2019|
|2020|
|2021|
|2022|
|2023|
|2024|
+----+
```

```
scala> corridors.select("origin_region").distinct().orderBy("origin_region").show(50,false)
+------------------+
|origin_region     |
+------------------+
|Andalucia         |
|Aragon            |
|Asturias          |
|Cantabria         |
|Castilla y Leon   |
|Castilla-La Mancha|
|Cataluna          |
|Com. Valenciana   |
|Extremadura       |
|Galicia           |
|Madrid            |
|Murcia            |
|Navarra           |
|Pais Vasco        |
|Rioja             |
+------------------+


scala> corridors.select("destination_region").distinct().orderBy("destination_region").show(50,false)
+------------------+
|destination_region|
+------------------+
|Andalucia         |
|Aragon            |
|Asturias          |
|Cantabria         |
|Castilla y Leon   |
|Castilla-La Mancha|
|Cataluna          |
|Com. Valenciana   |
|Extremadura       |
|Galicia           |
|Madrid            |
|Murcia            |
|Navarra           |
|Pais Vasco        |
|Rioja             |
+------------------+
```

```
scala> corridors.select("metric").distinct().show(50,false)
+--------------------------+
|metric                    |
+--------------------------+
|transport_operations_total|
|tonne_km_millions         |
|transport_operations_empty|
|tonnes_thousands          |
+--------------------------+


scala> corridors.columns.foreach { c => println(s"  $c: ${corridors.select(c).distinct().count()} distinc
t") }
  year: 6 distinct
  origin_region: 15 distinct
  destination_region: 15 distinct
  metric: 4 distinct
  value: 4839 distinct

scala> corridors.columns.foreach { c => println(s"  $c: ${corridors.filter(col(c).isNull || col(c)===""  |
| col(c)==="..").count()} nulls") }
  year: 0 nulls
  origin_region: 0 nulls
  destination_region: 0 nulls
  metric: 0 nulls
  value: 0 nulls

scala> corridors.select(min("value"),max("value"),avg("value")).show()
+----------+------------------+-----------------+
|min(value)|        max(value)|       avg(value)|
+----------+------------------+-----------------+
|1.20236368|2748015.6549999956|56756.83206720915|
+----------+------------------+-----------------+
```

```
scala> // 2. REGIONAL

scala> println("== REGIONAL ==")
== REGIONAL ==

scala> val regional = spark.read.option("header","true").option("inferSchema","true").csv(s"${hdfsBase}/r
egional")
regional: org.apache.spark.sql.DataFrame = [year: int, region: string ... 3 more fields]

scala> println(s"Total records: ${regional.count()}")
Total records: 2844

scala> regional.printSchema()
root
 |-- year: integer (nullable = true)
 |-- region: string (nullable = true)
 |-- displacement_type: string (nullable = true)
 |-- metric: string (nullable = true)
 |-- value: double (nullable = true)


scala> regional.rdd.map(r => (r.getAs[String]("displacement_type"), 1)).reduceByKey(_ + _).collect().sort
By(-_._2).foreach { case (k,v) => println(s"  $k -> $v") }
  intrarregional_total -> 432
  intramunicipal -> 432
  interregional_recibido -> 418
  interregional_expedido -> 417
  intermunicipal_intrarregional -> 408
  internacional_recibido -> 375
  internacional_expedido -> 362
```

```
scala> regional.select("region").distinct().orderBy("region").show(50,false)
+------------------+
|region            |
+------------------+
|Andalucia         |
|Aragon            |
|Asturias          |
|Baleares          |
|Canarias          |
|Cantabria         |
|Castilla y Leon   |
|Castilla-La Mancha|
|Cataluna          |
|Ceuta y Melilla   |
|Com. Valenciana   |
|Extremadura       |
|Galicia           |
|Madrid            |
|Murcia            |
|Navarra           |
|Pais Vasco        |
|Rioja             |
+------------------+


scala> regional.select("displacement_type").distinct().show(50,false)
+----------------------------+
|displacement_type           |
+----------------------------+
|internacional_recibido      |
|interregional_expedido      |
|intermunicipal_intrarregional|
|intramunicipal              |
|interregional_recibido      |
|intrarregional_total        |
|internacional_expedido      |
+----------------------------+
```

```
scala> regional.select("metric").distinct().show(50,false)
+-----------------+
|metric           |
+-----------------+
|operations_total |
|tonne_km_millions|
|operations_empty |
|tonnes_thousands |
+-----------------+


scala> regional.columns.foreach { c => println(s"  $c: ${regional.select(c).distinct().count()} distinct"
) }
  year: 6 distinct
  region: 18 distinct
  displacement_type: 7 distinct
  metric: 4 distinct
  value: 2814 distinct

scala> regional.columns.foreach { c => println(s"  $c: ${regional.filter(col(c).isNull || col(c)===("").co
unt()} nulls") }
  year: 0 nulls
  region: 0 nulls
  displacement_type: 0 nulls
  metric: 0 nulls
  value: 0 nulls
```

```
scala> // 3. CARGO

scala> println("== CARGO ==")
== CARGO ==

scala> val cargo = spark.read.option("header","true").option("inferSchema","true").csv(s"${hdfsBase}/carg
o")
cargo: org.apache.spark.sql.DataFrame = [year: int, region: string ... 4 more fields]

scala> println(s"Total records: ${cargo.count()}")
Total records: 3478

scala> cargo.printSchema()
root
 |-- year: integer (nullable = true)
 |-- region: string (nullable = true)
 |-- direction: string (nullable = true)
 |-- cargo_class: string (nullable = true)
 |-- metric: string (nullable = true)
 |-- value: double (nullable = true)


scala> cargo.rdd.map(r => (r.getAs[String]("cargo_class"), 1)).reduceByKey(_ + _).collect().sortBy(-_._2)
.foreach { case (k,v) => println(s"  $k -> $v") }
  Agricolas y animales vivos -> 360
  Alimenticios y forrajes -> 360
  Productos metalurgicos -> 360
  Productos petroliferos -> 360
  Minerales y materiales para construccion -> 360
  Maquinas, vehiculos, objetos manufacturados -> 360
  Productos quimicos -> 360
  Minerales y residuos para refundicion -> 358
  Abonos -> 358
  Combustibles minerales solidos -> 242
```

```
scala> cargo.select("cargo_class").distinct().orderBy("cargo_class").show(50,false)
+-----------------------------------------+
|cargo_class                              |
+-----------------------------------------+
|Abonos                                   |
|Agricolas y animales vivos               |
|Alimenticios y forrajes                  |
|Combustibles minerales solidos           |
|Maquinas, vehiculos, objetos manufacturados|
|Minerales y materiales para construccion |
|Minerales y residuos para refundicion    |
|Productos metalurgicos                   |
|Productos petroliferos                   |
|Productos quimicos                       |
+-----------------------------------------+


scala> cargo.select("direction").distinct().show(50,false)
+---------+
|direction|
+---------+
|recibido |
|expedido |
+---------+


scala> cargo.select("region").distinct().orderBy("region").show(50,false)
+------------------+
|region            |
+------------------+
|Andalucia         |
|Aragon            |
|Asturias          |
|Cantabria         |
|Castilla y Leon   |
|Castilla-La Mancha|
|Cataluna          |
|Com. Valenciana   |
|Extremadura       |
|Galicia           |
|Madrid            |
|Murcia            |
|Navarra           |
|Pais Vasco        |
|Rioja             |
+------------------+
```

```
scala> cargo.columns.foreach { c => println(s"  $c: ${cargo.select(c).distinct().count()} distinct") }
  year: 6 distinct
  region: 15 distinct
  direction: 2 distinct
  cargo_class: 10 distinct
  metric: 2 distinct
  value: 3467 distinct

scala> cargo.columns.foreach { c => println(s"  $c: ${cargo.filter(col(c).isNull || col(c)===("").count()}
 nulls") }
  year: 0 nulls
  region: 0 nulls
  direction: 0 nulls
  cargo_class: 0 nulls
  metric: 0 nulls
  value: 0 nulls

scala>

scala> // 4. PRICE INDEX

scala> println("== PRICE INDEX ==")
== PRICE INDEX ==

scala> val priceIndex = spark.read.option("header","true").option("inferSchema","true").csv(s"${hdfsBase}
/price_index")
priceIndex: org.apache.spark.sql.DataFrame = [year: int, quarter: string ... 2 more fields]

scala> println(s"Total records: ${priceIndex.count()}")
Total records: 144

scala> priceIndex.printSchema()
root
 |-- year: integer (nullable = true)
 |-- quarter: string (nullable = true)
 |-- distance_band: string (nullable = true)
 |-- price_index: double (nullable = true)
```

```
scala> priceIndex.rdd.map(r => (r.getAs[String]("distance_band"), 1)).reduceByKey(_ + _).collect().sortBy
(-_._2).foreach { case (k,v) => println(s"  $k -> $v") }
  Entre 201 y 300 km -> 24
  Entre 51 y 100 km -> 24
  50 km y menos -> 24
  Total -> 24
  Entre 101 y 200 km -> 24
  Mas de 300 km -> 24

scala> priceIndex.select("year").distinct().orderBy("year").show(50,false)
+----+
|year|
+----+
|2019|
|2020|
|2021|
|2022|
|2023|
|2024|
+----+


scala> priceIndex.select("quarter").distinct().orderBy("quarter").show(50,false)
+-------+
|quarter|
+-------+
|Q1     |
|Q2     |
|Q3     |
|Q4     |
+-------+


scala> priceIndex.select("distance_band").distinct().show(50,false)
+------------------+
|distance_band     |
+------------------+
|Entre 51 y 100 km |
|Total             |
|Entre 201 y 300 km|
|Mas de 300 km     |
|50 km y menos     |
|Entre 101 y 200 km|
+------------------+
```

```
scala> priceIndex.select(min("price_index"),max("price_index"),avg("price_index")).show()
+----------------+----------------+------------------+
|min(price_index)|max(price_index)|  avg(price_index)|
+----------------+----------------+------------------+
|           132.0|           182.4|156.01874999999998|
+----------------+----------------+------------------+
```

```
scala> // 5. DISTANCE BANDS

scala> println("== DISTANCE BANDS ==")
== DISTANCE BANDS ==

scala> val distBands = spark.read.option("header","true").option("inferSchema","true").csv(s"${hdfsBase}/
distance_bands")
distBands: org.apache.spark.sql.DataFrame = [year: int, distance_band: string ... 3 more fields]

scala> println(s"Total records: ${distBands.count()}")
Total records: 420

scala> distBands.printSchema()
root
 |-- year: integer (nullable = true)
 |-- distance_band: string (nullable = true)
 |-- service_type: string (nullable = true)
 |-- metric: string (nullable = true)
 |-- value: double (nullable = true)


scala> distBands.rdd.map(r => (r.getAs[String]("distance_band"), 1)).reduceByKey(_ + _).collect().sortBy(
-_._2).foreach { case (k,v) => println(s"  $k -> $v") }
  De 150 a 500 km -> 84
  Menos de 50 km -> 84
  Mas de 500 km -> 84
  Total -> 84
  De 50 a 149 km -> 84

scala> distBands.select("distance_band").distinct().show(50,false)
+---------------+
|distance_band  |
+---------------+
|Total          |
|Menos de 50 km |
|Mas de 500 km  |
|De 150 a 500 km|
|De 50 a 149 km |
+---------------+
```

```
scala> distBands.select("service_type").distinct().show(50,false)
+-------------+
|service_type |
+-------------+
|cuenta_ajena |
|total        |
|cuenta_propia|
+-------------+


scala> distBands.select("metric").distinct().show(50,false)
+-----------------+
|metric           |
+-----------------+
|operations_total |
|tonne_km_millions|
|operations_loaded|
|tonnes_thousands |
+-----------------+
```

```
scala> // 6. ACOTRAM

scala> println("== ACOTRAM ==")
== ACOTRAM ==

scala> val acotram = spark.read.option("header","true").option("inferSchema","true").csv(s"${hdfsBase}/ac
otram")
acotram: org.apache.spark.sql.DataFrame = [period: date, km_anual: double ... 13 more fields]

scala> println(s"Total records: ${acotram.count()}")
Total records: 23

scala> acotram.printSchema()
root
 |-- period: date (nullable = true)
 |-- km_anual: double (nullable = true)
 |-- pct_carga: double (nullable = true)
 |-- km_en_carga: double (nullable = true)
 |-- horas_anuales: double (nullable = true)
 |-- personal_conduccion_eur: double (nullable = true)
 |-- dietas_conductor_eur: double (nullable = true)
 |-- seguros_eur: double (nullable = true)
 |-- costes_fiscales_eur: double (nullable = true)
 |-- peajes_eur: double (nullable = true)
 |-- urea_eur: double (nullable = true)
 |-- costes_indirectos_eur: double (nullable = true)
 |-- fuel_price_per_litre: double (nullable = true)
 |-- fuel_consumption_coeff: double (nullable = true)
 |-- combustible_eur: double (nullable = true)


scala> acotram.withColumn("year",substring(col("period").cast("string"),1,4)).rdd.map(r=>(r.getAs[String]
("year"),1)).reduceByKey(_+_).collect().sortBy(_._1).foreach{case(k,v)=>println(s"  $k -> $v")}
  2019 -> 4
  2020 -> 4
  2021 -> 4
  2022 -> 4
  2023 -> 3
  2024 -> 4
```

```
scala> acotram.select("period").distinct().orderBy("period").show(50,false)
+----------+
|period    |
+----------+
|2019-01-31|
|2019-04-30|
|2019-07-31|
|2019-10-31|
|2020-01-31|
|2020-04-30|
|2020-07-31|
|2020-10-31|
|2021-01-31|
|2021-04-30|
|2021-07-31|
|2021-10-31|
|2022-01-31|
|2022-04-30|
|2022-07-31|
|2022-10-31|
|2023-01-31|
|2023-04-30|
|2023-10-31|
|2024-01-31|
|2024-04-30|
|2024-07-31|
|2024-10-31|
+----------+
```

```
scala> if (acotram.columns.contains("vehicle_type")) acotram.select("vehicle_type").distinct().show(50,fa
lse)

scala> acotram.select(min("fuel_price_per_litre"),max("fuel_price_per_litre"),avg("fuel_price_per_litre")
).show()
+-------------------------+-------------------------+-------------------------+
|min(fuel_price_per_litre)|max(fuel_price_per_litre)|avg(fuel_price_per_litre)|
+-------------------------+-------------------------+-------------------------+
|                   1.0183|                   1.9714|                   1.3895|
+-------------------------+-------------------------+-------------------------+


scala> Seq("personal_conduccion_eur","dietas_conductor_eur","seguros_eur","costes_fiscales_eur","combusti
ble_eur").foreach{c=>val s=acotram.select(min(c),max(c),avg(c)).first();println(s"  $c: min=${s.get(0)},
max=${s.get(1)}, avg=${s.get(2)}")}
  personal_conduccion_eur: min=29968.68, max=35648.51, avg=31643.982608695667
  dietas_conductor_eur: min=14247.87, max=16949.76, avg=15045.11086956522
  seguros_eur: min=6356.44, max=7661.58, avg=6639.166086956522
  costes_fiscales_eur: min=768.8, max=916.09, avg=820.1513043478261
  combustible_eur: min=7038.4896, max=14924.1504, avg=10246.929182608697

scala> acotram.columns.foreach{c=>println(s"  $c: ${acotram.filter(col(c).isNull||col(c)===("").count()} n
ulls")}
  period: 0 nulls
  km_anual: 0 nulls
  pct_carga: 0 nulls
  km_en_carga: 0 nulls
  horas_anuales: 0 nulls
  personal_conduccion_eur: 0 nulls
  dietas_conductor_eur: 0 nulls
  seguros_eur: 0 nulls
  costes_fiscales_eur: 0 nulls
  peajes_eur: 0 nulls
  urea_eur: 0 nulls
  costes_indirectos_eur: 0 nulls
  fuel_price_per_litre: 0 nulls
  fuel_consumption_coeff: 0 nulls
  combustible_eur: 0 nulls
```